

project-state-suite

Product Description

A company-wide work intelligence system for tracking the full breadth of active work — projects, prospects, operations, and concerns.

Atomic 47 Labs · May 2026
Version 1.0 — Internal Product Document

1. Executive Summary

project-state-suite is a structured intelligence system for tracking the full scope of a company's active work. It operates as a three-layer stack: a signal layer that harvests activity from everyday tools, an execution layer that manages individual projects in depth, and a portfolio layer called the docket that provides a company-wide view of every commitment, regardless of type or size.

The suite answers three questions that no single tool currently handles together:

- What is the company actually responsible for right now?
- How is each piece of work progressing?
- What is due, and to whom?

Where traditional project management tools track work inside a single project, project-state-suite tracks across all of them — including work that has not yet become a formal project (prospects), work with no defined end date (operational streams), and issues that span multiple initiatives (concerns).

2. The Problem

Most companies accumulate work faster than they can make it visible. A combination of email threads, spreadsheets, project tools, and institutional memory creates a picture of the portfolio that is always incomplete, always stale, and never in one place.

The consequences are predictable: deliverables fall through gaps between projects, prospects are not followed up, operational work becomes invisible until it fails, and leadership cannot answer the question "what are we working on" with confidence.

Existing tools are optimized for depth within a single project — Gantt charts, kanban boards, milestone trackers. None of them are optimized for breadth across an entire company's portfolio. That gap is what project-state-suite closes.

3. Vision and Design Intent

project-state-suite is designed around one core belief: that a company should be able to answer "what are we responsible for" in under thirty seconds, with confidence, at any time.

The suite is not a task manager. It is not a CRM. It is not a project management tool in the traditional sense. It is an intelligence layer that sits above those tools and synthesizes their outputs into a coherent company view.

Design Principles

- **Authority without duplication.** Each layer owns its data. The docket reads from the execution layer; it does not duplicate it. A milestone updated in a project file is reflected in the docket automatically.
- **Frictionless intake.** The system is only as useful as the discipline required to keep it current. Intake must be as fast as saying it out loud. Entries require only a name, a type, and an owner to be created.
- **YAML as source of truth.** All state lives in human-readable YAML files. The system is diffable, portable, and not dependent on any proprietary platform. A dashboard is a view of the files, not the other way around.
- **Separation of concerns.** Signal flows up, authority flows down. The docket does not write back to project files. Activity logs do not write back to the docket. Information moves in one direction.
- **Progressive disclosure.** A small entry in the docket can become a fully managed project with milestones, documents, and funder reporting without changing its identity. The system grows with the work.

4. Architecture: The Three-Layer Stack

project-state-suite is organized into three distinct layers. Each has a clear purpose, a clear data owner, and a clear relationship to the layers above and below it.

Layer	Name	Purpose	Answers
-------	------	---------	---------

1 — Signal	work-state	Harvests activity from GitHub, Gmail, Slack, Google Drive into a personal activity log.	What has been touched?
2 — Execution	project-state	Manages individual projects in depth: milestones, phases, decisions, risks, funder reporting.	How is this project going?
3 — Portfolio	docket-state	Company-wide register of all work: projects, prospects, operational streams, concerns.	What are we responsible for?

Information flows upward only. work-state feeds activity signal to docket-state. project-state feeds health and milestone data to docket-state. The docket does not write back down to either.

Layer 1: Signal (work-state)

work-state is the personal activity intelligence layer. It harvests events from connected tools — commits from GitHub, messages from Gmail and Slack, edits from Google Drive — and stores them in a structured activity log at `~/.work-state/`. It answers questions about what has been touched and by whom. It does not interpret; it records.

Layer 2: Execution (project-state)

project-state is the per-project management layer. Each project lives in a `.project-state/` directory containing a manifest, milestone records, decision logs, risk registers, document indexes, and a project-specific activity log. It is designed for projects with defined scope and governance requirements: funder reporting, milestone tracking, steering committee management, IP disclosure, and phase-gate transitions.

A project does not need a `.project-state/` directory to appear on the docket. project-state is the execution layer for projects that warrant that level of management. Smaller engagements, prospects, and operational work live in the docket layer only.

Layer 3: Portfolio (docket-state)

docket-state is the company-wide portfolio layer. It is a structured register of everything the company is responsible for, organized into four entry types. It reads from project-state instances to show health and milestone status, and adds categories of work that project-state does not model. The docket is the primary surface for understanding the breadth of the company's work.

5. The Docket: Entry Types

Every entry in the docket is one of four types. The type determines how the entry is tracked, what fields it carries, and what lifecycle transitions are available.

5.1 Project

A project has a defined scope, a defined end, and at least one owner. If a project has a `.project-state/` directory, the docket entry links to it and reads health and milestone data automatically. If not, the project lives in the docket as a lightweight record.

Field	Description
name	Short identifier. Used in all views and references.
owner	Primary accountable person.
status	active / at-risk / on-hold / complete
project_state_path	Optional. Path to <code>.project-state/</code> directory. If present, health and milestones are read automatically.
pending_deliverables	List of upcoming deliverables with due dates. Rolled up into the company deliverables view.
health	green / amber / red. Set manually or derived from project-state if linked.

5.2 Prospect

A prospect is pre-commitment work: a bid, a conversation, an opportunity that has not yet become a formal project. Prospects are tracked through a simple stage model. When a prospect is won, it is promoted to a project entry. When lost or closed, it is archived with a note.

Field	Description
stage	early / active / proposal / decision / closed
contact	Primary external contact or organization.
next_action	Single next step with owner and due date.
value	Optional estimated value or scope. Used for portfolio weighting.
outcome	Populated on close: won / lost / no-decision, with a brief note.

5.3 Operational Stream

An operational stream is ongoing work with no defined end date. Infrastructure, support, recurring services, maintenance programs — work that keeps the company running but does not fit the project model. Operational streams are tracked by health and current open items rather than milestones.

Field	Description
cadence	How often this stream produces work or requires review (daily / weekly / monthly / as-needed).
health	green / amber / red. Primary indicator for operational streams.
open_items	Current list of open items or issues within this stream.
last_reviewed	Date of last health review. Streams not reviewed within their cadence are flagged.

5.4 Concern

A concern is a risk, issue, or pattern that does not fit cleanly inside one project. A resourcing pressure that touches three projects is one concern, not three separate risk records. Concerns are cross-cutting by definition and can reference any combination of projects, prospects, or operational streams.

Field	Description
severity	low / medium / high / critical
affects	List of docket entry IDs that this concern touches.

owner	Person accountable for resolution.
resolution	Populated when closed. Brief description of how the concern was resolved.

6. Entry Lifecycle

Docket entries move through a defined lifecycle. The lifecycle is designed to be simple enough that it can be maintained in conversation — advancing a prospect to a project or closing a concern should take seconds, not a workflow.

```
prospect ► project ► operational stream
      |           |
      |   (if scope is ongoing with no close date)
      |
      ► closed  (won / lost / no-decision)

concern ► resolved

operational stream ► deprecated
```

Any entry type can be closed or archived. Closed entries are retained for historical reference but do not appear in active views. Concerns are resolved, not closed — the distinction signals that resolution requires an action, not just a status change.

7. Docket Views

The docket produces two primary views, both generated from YAML source files and renderable in a browser, as a printed document, or embedded in another tool.

7.1 Company Board

The company board is the primary operational view. It shows all docket entries organized by type, with health indicators, owners, and a count of pending deliverables. The board is designed to be readable in under two minutes by anyone in the company.

The board surfaces three categories of attention: entries with red or critical health requiring immediate action, entries with deliverables due in the next 14 days, and entries that have not been reviewed within their expected cadence.

7.2 Deliverables Roll-Up

The deliverables roll-up is a flat, date-sorted list of every pending deliverable across all active docket entries — regardless of which project, prospect, or operational stream they belong to. It is the single answer to the question "what is due and when."

Deliverables are pulled from three sources: `pending_deliverables` on project entries, `next_action` fields on prospects (treated as deliverables with a due date), and `open_items` on operational streams that carry a due date.

8. The Skill Suite

project-state-suite is implemented as a set of Claude skills — specialized agents invoked in conversation or automated on a schedule. Skills are organized by layer.

8.1 Signal Layer (work-state)

Skill	Function
work-state	Shared memory of the work intelligence facility. Reads, writes, and validates <code>~/.work-state/</code> .
work-orchestrator	Decides what to harvest and synthesize next based on current state and cadence.
work-harvester-github	Harvests commits, pull requests, and releases from GitHub.
work-harvester-gmail	Harvests sent and received messages from Gmail.
work-harvester-slack	Harvests messages from Slack channels and DMs.
work-harvester-gdocs	Harvests edits, creates, and shares from Google Docs.
work-kanban	Local Next.js dashboard rendering work-state as a portfolio kanban with activity sparklines.

8.2 Execution Layer (project-state)

Skill	Function
project-state	Shared memory of a project. Reads, writes, and validates .project-state/ directories.
project-scaffolder	Initializes a new .project-state/ facility for a new project.
project-orchestrator	Decides what to do next based on state, calendar, and deadlines. The daily briefing skill.
project-milestone-manager	Creates, updates, and reports on milestones. Tracks percent complete and technical progress.
project-phase-gate	Manages lifecycle phase transitions. Enforces gate checklists and blocks premature transitions.
project-document-curator	Classifies, indexes, and manages project documents. Maintains document source-of-truth.
project-funder-reporting	Generates funder and stakeholder reports. Configurable for different reporting frameworks.
project-review-meeting	Full lifecycle for recurring review meetings: scheduling, packs, minutes, action items.
project-change-register	Registers and classifies changes. Distinguishes material changes from non-material.
project-ip-tracker	Tracks intellectual property arising from projects. Routes disclosures to configured recipients.
project-lessons	Captures lessons learned continuously from kickoff through close.
project-notifier	Routes artifacts to Slack, Gmail drafts, or Google Calendar.
project-kanban	Local Next.js dashboard for a single .project-state/ directory.

8.3 Portfolio Layer (docket-state)

The docket layer skills are the newest addition to the suite and represent the active development frontier.

Skill	Function
docket-state	Shared memory of the company docket. Reads, writes, and validates ~/.docket-state/.
docket-intake	Adds and updates docket entries. Minimal friction — name, type, owner is enough to start.

docket-reporter	Generates the deliverables roll-up and company board views.
docket-orchestrator	Morning briefing skill. Surfaces what needs attention based on health, cadence, and due dates.
docket-kanban	Local Next.js dashboard rendering the company board as a live kanban view.

9. Information Flows

Understanding how information moves through the stack is essential to using the suite correctly and extending it without introducing conflicts.

What flows upward

- work-state → docket-state: Activity signal. Which projects are being actively touched, who is working on what, velocity indicators.
- project-state → docket-state: Health status, milestone completion percentages, pending deliverables, current phase.

What does not flow

- docket-state does not write to project-state. The docket is a read-only consumer of project data.
- work-state does not write to docket-state directly. The orchestrator synthesizes activity signals into docket-readable summaries on demand.
- No layer writes to the tools it harvests from. Harvesting is strictly read-only.

Authority model

Each layer owns its data. project-state is the authority for how a specific project is going. docket-state is the authority for what the company is responsible for. work-state is the authority for what has been touched and when. When they appear to conflict, the more granular layer wins: project-state overrides the docket, and the docket entry is updated to reflect it.

10. Extensibility

project-state-suite is designed to grow with the company. YAML-backed state means new fields can be added to any entry type without breaking existing data. New skills can be added to any layer without affecting the others.

Pack system

The execution layer supports packs — configuration profiles that adapt skill behavior for a specific funder, customer type, or governance framework. A PIC grant pack configures quarterly claim reporting, steering committee cadence, and IP disclosure routing. A client-services pack configures customer invoicing, QBR scheduling, and deliverable sign-off. New packs can be authored without modifying the core skills.

Multi-contributor support

docket-state is designed for team use. Entries carry owner fields, and the kanban view groups by owner for individual workload views. YAML files are designed to be committed to a shared repository, enabling the full team to read and update the docket from their own environments.

Connector integrations

The suite integrates with external services through MCP connectors: GitHub for commit harvesting, Gmail for email harvesting and draft creation, Slack for notifications, and Google Drive for document activity. Additional connectors can be added as new harvester skills without affecting the rest of the suite.

11. Roadmap

The following capabilities are planned or in active development.

Near-term (docket layer completion)

- Scaffold docket-state directory structure and manifest schema

- Implement docket-intake with minimal-friction entry creation
- Implement docket-reporter for company board and deliverables roll-up
- Build docket-kanban as a local Next.js dashboard
- Implement docket-orchestrator for morning briefing
- Connect project-state health reads to docket entry display

Medium-term

- Multi-contributor docket with shared repository sync
- Prospect lifecycle with conversion tracking and outcome logging
- Cross-entry concern detection: automatic surfacing of risks spanning multiple entries
- Deliverables calendar export: push the roll-up to Google Calendar
- Docket digest: weekly email summary of portfolio health and upcoming deliverables

Longer-term

- Portfolio analytics: trend lines on health, velocity, and deliverable completion rate
- Prospect pipeline analytics: conversion rates, stage duration, win/loss patterns
- Integration with time tracking to map effort against docket entries
- Natural language briefing: "What should I know about the portfolio this week?" answered in prose

12. Glossary

Term	Definition
docket	The company-wide register of all active work. The primary output of docket-state.
docket entry	A single record in the docket: one project, prospect, operational stream, or concern.
project-state	The per-project management layer. Lives in a .project-state/ directory inside a project folder.
work-state	The personal activity intelligence layer. Lives at ~/.work-state/.
docket-state	The portfolio intelligence layer. Lives at ~/.docket-state/.

pack	A configuration profile that adapts skill behavior for a specific funder, customer, or governance framework.
signal layer	Layer 1. Harvests raw activity from connected tools.
execution layer	Layer 2. Manages individual projects in depth.
portfolio layer	Layer 3. Company-wide breadth view of all active work.
deliverables roll-up	A flat, date-sorted list of all pending deliverables across all docket entries.
company board	The primary docket view: all entries by type, with health, owner, and pending deliverable count.
harvester	A skill that reads activity from a connected external tool and writes events to work-state.
health	A three-value indicator (green / amber / red) summarizing whether an entry is on track.

project-state-suite

Atomic 47 Labs · May 2026 · Version 1.0